

AI_reference — Firebase 聊天室期中專題

SECTION 1 : AI Tool(s) Used

本專案開發過程中使用了 Claude (Anthropic) 作為主要 AI 輔助工具，模型版本為 Claude Sonnet。

SECTION 2 : Scope of Usage

以下列出本專案中有使用 AI 輔助的主要段落，並說明使用的 prompt、AI 的回應內容，以及我自己最後修改的版本與原因。

SECTION 2-1 : 雙向封鎖功能 (Bidirectional Block)

檔案：public/js/chat.js，約第 51–92 行 (findLastVisibleMsg)、第 334–353 行 (renderMessage 封鎖過濾)

問題背景：

原本封鎖功能只有單向，也就是我封鎖對方，但對方還是看得到我的訊息。老師的評分標準要求雙向封鎖，而且封鎖之前的訊息要繼續可見，只有封鎖之後的訊息才需要隱藏。這個 timestamp 機制我想了很久不知道怎麼實作比較好，就問了 AI。

使用的 Prompt (大意)：

「我要實作雙向封鎖，封鎖後的訊息要隱藏，封鎖前的訊息要保留，要怎麼用 timestamp 來判斷？」

AI 的回應重點：

AI 建議在 Firestore 的 user document 裡新增一個 blockedAt 欄位，格式是 { uid: timestamp }，記錄封鎖的時間點。每次渲染訊息時，比較訊息的 createdAt.seconds 和 blockedAt[senderId]，如果訊息時間比封鎖時間晚就隱藏，比封鎖時間早就繼續顯示。另外對方封鎖我的情況，要另外抓對方的 user document 來取得他的 blockedAt，才能判斷他封鎖我的時間點。

我的修改與說明：

AI 給的邏輯基本上是對的，但我發現 renderMessage 裡面只處理了「我封鎖別人」的情況，沒有處理「別人封鎖我」的情況。所以我額外加了 blockedBySet 和 blockedByAt 這兩個全域變數，在進入聊天室 (openRoom) 的時候先把所有成員的封鎖狀態抓好看起來，這樣 renderMessage 就可以直接查這兩個變數，不用每次都去打 Firestore。另外側欄的訊息預覽也要做同樣的處理，AI 寫了 findLastVisibleMsg 這個 function，會從最新訊息往前掃，跳過被封鎖的訊息，找到第一則可見的訊息再回傳。

SECTION 2-2 : 側欄聊天室列表不閃爍 (Incremental Update)

檔案：public/js/chat.js，約第 159–200 行 (loadRooms)

問題背景：

原本每次有新訊息進來，側欄的聊天室列表就會整個清掉重新渲染，畫面會一直閃。我想改成只更新有變動的項目，但不知道怎麼用 Firestore 的 snapshot 做到這件事。

使用的 Prompt (大意)：

「側欄每次收到訊息都會整個 re-render 然後閃爍，要怎麼讓它只更新有改變的那一個聊天室就好？」

AI 的回應重點：

AI 說要改用 `snapshot.docChanges()` 來取代直接用整個 snapshot，`docChanges()` 會回傳一個陣列，裡面每一個 change 都有 type (added / modified / removed)。只有 added 的時候才新增 DOM 元素，modified 的時候只更新那個元素的文字內容，removed 的時候才刪掉。

我的修改與說明：

AI 給的架構是對的，但 modified 的時候 AI 原本是用 `replaceWith` 換掉整個元素，這樣還是會觸發 CSS 動畫導致閃爍。我改成只更新 `.room-name` 和 `.room-last-msg` 的 `textContent`，這樣 DOM 元素本身不動，就不會有動畫重播的問題。

SECTION 2-3：群組聊天室封鎖通知問題

檔案：public/js/chat.js，約第 215–248 行 (setupRoomNotification)

問題背景：

在群組裡，如果 A 封鎖了 B，B 傳訊息的時候 A 還是會收到通知。這個問題出在原本的通知邏輯用 `forEach` 加上 `await`，但 `forEach` 不支援 `async`，所以 `await` 根本沒有等待，全部都直接跑過去了。

使用的 Prompt (大意)：

「群組封鎖後還是會跳通知，我懷疑是 `forEach` 跟 `await` 的問題，要怎麼修？」

AI 的回應重點：

AI 確認問題就是 `forEach` 不能搭配 `await`，要改成 `for...of` 迴圈才能讓 `await` 正常運作。同時 AI 也補充說要加一個 `senderHasBlockedMe()` 的 helper function，去 Firestore 抓發送者的資料，確認他有沒有封鎖我，有的話就跳過不通知。

我的修改與說明：

這段基本上照 AI 給的改，把 `forEach` 換成 `for...of`，然後加上 `senderHasBlockedMe` 的判斷。邏輯沒有什麼要改的地方，主要就是 JavaScript 非同步的語法問題。

SECTION 2-4：AI 聊天機器人 API 除錯過程

檔案：public/js/chat.js，約第 1133–1217 行 (AI Chatbot)

問題背景：

這個功能卡了很久，主要是 API 一直報錯。一開始用 Gemini，試了好幾個 endpoint 和 model 名稱都不對，後來才發現是因為這個 API key 所在的 Google Cloud project 對那些 model 的 free tier quota 是 0，根本不能用。

除錯過程：

- 第一次嘗試用 v1/models/gemini-1.5-flash，報錯說 v1 找不到這個 model。
- 改成 v1beta/models/gemini-1.5-flash，還是報錯說 v1beta 也找不到。
- 換了一把新的 Gemini API key 再試，同樣的錯誤。
- 後來用瀏覽器直接打 ListModels endpoint 查這個 key 有哪些 model 可以用，發現根本沒有 gemini-1.5-flash。
- 換成 gemini-2.0-flash 之後終於找到 model 了，但這次換成 quota exceeded，free tier limit 是 0。

使用的 Prompt（大意）：

「Gemini 一直 quota 0 用不了，有沒有其他免費的 AI API 可以用？」

AI 的回應重點：

AI 推薦改用 Groq，說它有非常寬鬆的免費方案，不需要信用卡，而且支援 OpenAI 相容的 API 格式，改起來很快。

最終使用的程式碼：

```
const res = await fetch("https://api.groq.com/openai/v1/chat/completions", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${GROQ_API_KEY}`
  },
  body: JSON.stringify({
    model: "llama-3.1-8b-instant",
    messages: [{ role: "user", content: userText }]
  })
});
const botReply = data.choices?.[0]?.message?.content
  || "Sorry, I couldn't get a response.";
```

說明：

原本 Gemini 用的是 contents[].parts[].text 的格式，Groq 用的是 OpenAI 的 messages[].content 格式。另外第一個 Groq model llama3-8b-8192 也被淘汰了，改用 llama-3.1-8b-instant。

SECTION 2-5：回覆訊息功能（Reply to Message）

檔案：public/js/chat.js，約第 978–999 行、public/css/chat.css reply 相關樣式

問題背景：

這個功能要在訊息泡泡裡顯示被引用的訊息，同時輸入框上方要有預覽列。資料結構和 UI 邏輯我知道大概怎麼做，但請 AI 幫我把整體架構寫出來再自己調整。

使用的 Prompt（大意）：

「幫我實作回覆訊息功能，右鍵選單要有 Reply，點了之後輸入框上面出現預覽，傳送的訊息要帶 replyTo 欄位，顯示的時候在泡泡裡面顯示被引用的內容。」

AI 的回應重點：

AI 設計了一個 `replyToMsg` 全域變數存目前要回覆的訊息，`setReplyTo(msg)` 負責設定這個變數並顯示預覽列，傳送時把 `replyTo: { id, text, senderName }` 存進 Firestore，`renderMessage` 裡面根據有沒有 `replyTo` 欄位來決定要不要顯示引用區塊。

我的修改與說明：

整體架構沒有改，主要調整了 CSS 的樣式讓引用區塊看起來比較自然，加了左側的彩色邊框和灰色背景。另外 AI 原本 `Reply` 只對別人的訊息顯示，我改成自己的訊息也可以按 `Reply`，比較符合一般聊天軟體的使用習慣。

SECTION 3 : Statement of Non-Usage

本專案有使用 AI 工具，詳細內容如上所述。